

UNIUBE

HANDY

DOCUMENTAÇÃO DE SOFTWARE

Felipe Lino

Guilherme Garcia

Guilherme Antônio De Souza

Haristoneu Garcia

Melissa Martins

Uberlândia — MG

2026

Felipe Lino
Guilherme Garcia
Guilherme Antônio De Souza
Haristoneu Garcia
Melissa Martins

HANDY — DOCUMENTAÇÃO DE SOFTWARE

Prof. Romualdo

Uberlândia — MG
2026

SUMÁRIO

1	INTRODUÇÃO	3
2	VISÃO GERAL DO SISTEMA.....	4
3	ARQUITETURA DO SISTEMA	5
3.1	Stack Tecnológica.....	6
4	BANCO DE DADOS	6
4.1	Descrição das Entidades	7
4.1.1	Entidade: usuario.....	7
4.1.2	Entidade: cliente	8
4.1.3	Entidade: prestador	8
4.1.4	Entidade: especialidade	8
4.1.5	Entidade: prestador_especialidade	9
4.1.6	Entidade: categoria.....	9
4.1.7	Entidade: servicos	10
4.1.8	Entidade: contratacoes.....	10
4.1.9	Entidade: pagamento	11
4.1.10	Entidade: conversa.....	11
4.1.11	Entidade: mensagem	12
4.1.12	Entidade: avaliacao	13
4.2	Regras e Restrições	13
4.2.1	Restrições de Domínio (CHECK).....	14
4.2.2	Integridade Referencial	14
4.2.3	Unicidade	15
4.3	Funções e Triggers	15
4.4.1	fn_atualizar_updated_at.....	16
4.4.2	fn_atualizar_media_prestador.....	16
4.4.3	fn_atualizar_total_clientes_prestador	17
4.4.4	finalizar_contratacao	17
4.5	Views	18
4.6	Índices	19
5	REFERÊNCIAS	19

1 INTRODUÇÃO

O presente documento constitui a documentação técnica de software do projeto Handy, uma plataforma digital voltada à conexão entre clientes que necessitam de serviços domésticos ou profissionais especializados e prestadores de serviço disponíveis. Esta documentação foi elaborada em conformidade com as normas da Associação Brasileira de Normas Técnicas (ABNT) e tem por objetivo registrar, de forma estruturada e progressiva, as decisões técnicas, arquiteturas e de implementação adotadas ao longo do ciclo de desenvolvimento.

A documentação está organizada em capítulos que cobrem distintos aspectos do sistema. Na versão atual, são apresentados a visão geral do produto, a arquitetura tecnológica adotada e o modelo de dados relacional, incluindo a descrição detalhada das entidades, as regras de integridade, as funções, os triggers e as visões implementadas no banco de dados PostgreSQL. Novos capítulos serão incorporados a este documento à medida que outras camadas do sistema forem desenvolvidas e documentadas.

2 VISÃO GERAL DO SISTEMA

O Handy é um sistema composto por uma aplicação web, um aplicativo mobile e uma API backend, oferecendo funcionalidades de busca, contratação, comunicação e gerenciamento de serviços em um único ambiente centralizado. A plataforma visa solucionar quatro problemas identificados na contratação de serviços profissionais: a ausência de plataformas centralizadas, a dificuldade em verificar a confiabilidade de profissionais, a falta de transparência quanto a preços e disponibilidade, e a comunicação ineficiente entre as partes durante o processo de contratação.

Os principais perfis de usuário são o cliente, que busca e contrata serviços, e o prestador, que os oferece. O fluxo central do sistema compreende: o cadastro e autenticação de usuários; a busca e filtragem de prestadores por categoria e localidade; a contratação de serviços com registro de detalhes, endereço e forma de pagamento; a comunicação via chat integrado; o processamento de pagamentos por meio de integração com a plataforma Stripe; e a avaliação do prestador ao término de cada contratação.

O escopo inclui a aplicação web (React + Vite), o aplicativo mobile (React Native), a API backend em NestJS com banco de dados PostgreSQL hospedado na Amazon RDS, e a infraestrutura de deploy na AWS. Estão fora do escopo o suporte a múltiplos idiomas e integrações com sistemas legados externos.

3 ARQUITETURA DO SISTEMA

O sistema Handy adota uma arquitetura monolítica modular, organizada em camadas bem definidas: apresentação (frontend web e mobile), lógica de negócio (backend API) e persistência de dados (banco de dados relacional). A comunicação entre as camadas ocorre por meio de uma API RESTful, o que preserva o desacoplamento entre frontend e backend e facilita a evolução independente de cada camada. A infraestrutura é hospedada na Amazon Web Services (AWS), com instâncias EC2 para o servidor de aplicação e Amazon RDS para o banco de dados PostgreSQL.

3.1 Stack Tecnológica

A Tabela 1 apresenta as tecnologias adotadas em cada camada do sistema.

Camada	Tecnologias
Frontend	TypeScript · React · Vite · Shadcn UI · Zod · React Hook Form
Backend	TypeScript · NestJS · Prisma ORM · JWT · Argon 2 · Stripe · OAuth
Banco de Dados	PostgreSQL
Mobile	TypeScript · React Native · Zod · React Hook Form
Infraestrutura	AWS · Arquitetura Monolítica Modular

Tabela 1 — Stack tecnológica do projeto Handy

4 BANCO DE DADOS

O modelo de dados do Handy é implementado em PostgreSQL e organizado em torno de doze entidades principais. O esquema foi projetado para garantir integridade referencial, evitar redundância e suportar as principais operações do domínio do negócio, como contratação, pagamento, comunicação e avaliação de prestadores. As seções a seguir descrevem o diagrama de entidade-relacionamento, as entidades e seus atributos, as restrições de negócio, os triggers e as visões disponíveis.

4.1 Descrição das Entidades

As subseções a seguir detalham cada entidade do modelo de dados, descrevendo sua finalidade e os atributos que a compõem.

4.1.1 Entidade: usuario

Entidade central que armazena os dados comuns a todos os usuários da plataforma, independentemente do perfil (cliente ou prestador). Campos como email e CPF possuem restrição de unicidade. O campo tipo_usuario distingue o perfil do usuário no sistema.

Campo	Tipo	Descrição
user_id	SERIAL	PK — Identificador único, gerado automaticamente
nome	VARCHAR(100)	Nome completo do usuário (obrigatório)
email	VARCHAR(100)	Endereço de e-mail único (obrigatório)
hash_password	TEXT	Senha armazenada com hash criptográfico
cpf	VARCHAR(14)	CPF do usuário (único, opcional)
tipo_usuario	VARCHAR(20)	Perfil: 'cliente' ou 'prestador'
photo_url	TEXT	URL da foto de perfil
endereco	TEXT	Endereço residencial ou comercial
descricao	TEXT	Biografia ou descrição do perfil
created_at	TIMESTAMP	Data e hora de criação do registro

updated_at	TIMESTAMP	Data e hora da última atualização (trigger automático)
------------	-----------	--

Tabela 2 — Campos da entidade usuario

4.1.2 Entidade: cliente

Entidade especializada para o perfil de cliente. Herda os dados gerais de usuario por meio de uma relação de herança por referência, utilizando user_id como chave primária e estrangeira simultaneamente. A exclusão do usuário pai propaga-se em cascata.

Campo	Tipo	Descrição
user_id	INT	PK/FK — Referencia usuario(user_id), com CASCADE

Tabela 3 — Campos da entidade cliente

4.1.3 Entidade: prestador

Entidade especializada para o perfil de prestador de serviços. Além de herdar os dados de usuario, armazena métricas calculadas automaticamente por triggers: a média das avaliações recebidas e o total de clientes atendidos (contratações concluídas).

Campo	Tipo	Descrição
user_id	INT	PK/FK — Referencia usuario(user_id), com CASCADE
media_avaliacao	NUMERIC(3,2)	Média ponderada das avaliações (0–5), atualizada por trigger
total_clientes	INT	Total de clientes com contratações concluídas, atualizado por trigger

Tabela 4 — Campos da entidade prestador

4.1.4 Entidade: especialidade

Catálogo de especialidades que podem ser associadas a prestadores. Funciona como um domínio controlado de competências técnicas específicas.

Campo	Tipo	Descrição
especialidade_id	SERIAL	PK — Identificador único
nome_especialidade	VARCHAR(100)	Nome da especialidade (único, obrigatório)

Tabela 5 — Campos da entidade especialidade

4.1.5 Entidade: prestador_especialidade

Tabela de junção que implementa a relação muitos-para-muitos entre prestadores e especialidades. Um prestador pode possuir múltiplas especialidades, e uma mesma especialidade pode ser atribuída a vários prestadores.

Campo	Tipo	Descrição
prestador_id	INT	PK/FK — Referencia prestador(user_id)
especialidade_id	INT	PK/FK — Referencia especialidade(especialidade_id)

Tabela 6 — Campos da entidade prestador_especialidade

4.1.6 Entidade: categoria

Catálogo de categorias de serviços disponíveis na plataforma. Cada serviço é obrigatoriamente vinculado a uma categoria, e a exclusão de uma categoria com serviços associados é bloqueada (ON DELETE RESTRICT).

Campo	Tipo	Descrição
categoria_id	SERIAL	PK — Identificador único
nome_categoria	VARCHAR(100)	Nome da categoria (único, obrigatório)
rank_categoria	INT	Peso para ordenação e destaque na interface

Tabela 7 — Campos da entidade categoria

4.1.7 Entidade: servicos

Representa os serviços cadastrados por prestadores. Cada serviço está vinculado a exatamente um prestador e a exatamente uma categoria. O preço não pode ser negativo (restrição CHECK).

Campo	Tipo	Descrição
servico_id	SERIAL	PK — Identificador único
prestador_id	INT	FK — Referencia prestador(user_id), com CASCADE
categoria_id	INT	FK — Referencia categoria(categoria_id), com RESTRICT
nome_servico	VARCHAR(150)	Nome descritivo do serviço
descricao	TEXT	Detalhamento do serviço
preco	NUMERIC(10,2)	Preço base (≥ 0)

Tabela 8 — Campos da entidade servicos

4.1.8 Entidade: contratacoes

Núcleo transacional do sistema. Registra cada solicitação de serviço realizada por um cliente a um prestador, incluindo status do ciclo de vida, dados financeiros, datas e endereço de execução. O campo status é controlado por um conjunto de valores permitidos.

Campo	Tipo	Descrição
contratacao_id	SERIAL	PK — Identificador único
cliente_id	INT	FK — Referencia cliente(user_id)
prestador_id	INT	FK — Referencia prestador(user_id)
servico_id	INT	FK — Referencia servicos(servico_id)
titulo	VARCHAR(100)	Título da contratação

detalhes	TEXT	Descrição detalhada do serviço solicitado
valor	NUMERIC(10,2)	Valor acordado (≥ 0)
endereco	TEXT	Local de execução do serviço
forma_pagamento	VARCHAR(20)	Pix Dinheiro Boleto Cartão
status	VARCHAR(20)	Pendente Aceita Em_Andamento Concluída Cancelada
inicio	TIMESTAMP	Data/hora de início da execução
conclusao	TIMESTAMP	Data/hora de conclusão
vencimento	TIMESTAMP	Prazo limite para aceitação ou execução
created_at/updated_at	TIMESTAMP	Datas de criação e atualização (trigger automático)

Tabela 9 — Campos da entidade contratacoes

4.1.9 Entidade: pagamento

Registra as transações financeiras associadas a contratações. O campo `plataforma_id_pagamento` armazena o identificador gerado pela plataforma Stripe, permitindo rastreabilidade e estorno. Uma contratação pode possuir múltiplos registros de pagamento (tentativas, estornos).

Campo	Tipo	Descrição
<code>pagamento_id</code>	SERIAL	PK — Identificador único
<code>contratacao_id</code>	INT	FK — Referencia contratacoes(<code>contratacao_id</code>)
<code>plataforma_id_pagamento</code>	VARCHAR(200)	ID externo gerado pelo Stripe
valor	NUMERIC(10,2)	Valor da transação (≥ 0)
status	VARCHAR(20)	pendente pago falhou cancelado estornado
<code>data_pagamento</code>	TIMESTAMP	Data/hora da confirmação do pagamento

Tabela 10 — Campos da entidade pagamento

4.1.10 Entidade: conversa

Representa o canal de comunicação vinculado a uma contratação. Cada contratação possui exatamente uma conversa (UNIQUE em contratacao_id). A conversa pode estar aberta, fechada ou arquivada.

Campo	Tipo	Descrição
conversa_id	SERIAL	PK — Identificador único
contratacao_id	INT	FK/UNIQUE — Uma conversa por contratação
cliente_id	INT	FK — Referencia cliente(user_id)
prestador_id	INT	FK — Referencia prestador(user_id)
status	VARCHAR(20)	aberta fechada arquivada

Tabela 11 — Campos da entidade conversa

4.1.11 Entidade: mensagem

Armazena as mensagens trocadas em uma conversa. Cada mensagem pertence a uma conversa e é enviada por um usuário identificado pelo campo remetente_id. O ciclo de vida de cada mensagem é rastreado pelo campo status.

Campo	Tipo	Descrição
mensagem_id	SERIAL	PK — Identificador único
conversa_id	INT	FK — Referencia conversa(conversa_id)
remetente_id	INT	FK — Referencia usuario(user_id)
conteudo	TEXT	Texto da mensagem
remetente_tipo	VARCHAR(20)	cliente prestador
status	VARCHAR(20)	enviada recebida lida
anexo_url	TEXT	URL de arquivo anexado (opcional)

Tabela 12 — Campos da entidade mensagem

4.1.12 Entidade: avaliacao

Registra a avaliação feita pelo cliente ao término de uma contratação. Uma contratação só pode ser avaliada uma vez (UNIQUE em contratacao_id). A nota deve estar entre 1 e 5 (CHECK). A inserção, atualização ou exclusão de uma avaliação dispara automaticamente o recálculo da média do prestador.

Campo	Tipo	Descrição
avaliacao_id	SERIAL	PK — Identificador único
contratacao_id	INT	FK/UNIQUE — Uma avaliação por contratação
prestador_id	INT	FK — Referencia prestador(user_id)
cliente_id	INT	FK — Referencia cliente(user_id)
nota	INT	Nota de 1 a 5 (CHECK)
comentario	TEXT	Comentário textual (opcional)
created_at	TIMESTAMP	Data da avaliação

Tabela 13 — Campos da entidade avaliacao

4.2 Regras e Restrições

O modelo de dados implementa diversas restrições de integridade diretamente no banco de dados, garantindo a consistência dos dados independentemente da camada de aplicação que os manipula. As principais restrições são descritas a seguir.

4.2.1 Restrições de Domínio (CHECK)

As restrições CHECK garantem que certos campos aceitem apenas valores pertencentes a um conjunto predefinido ou dentro de um intervalo válido. A Tabela 14 lista as restrições de domínio implementadas.

Tabela	Campo	Valores Permitidos
contratacoes	status	Pendente, Aceita, Em_Andamento, Concluída, Cancelada
contratacoes	forma_pagamento	Pix, Dinheiro, Boleto, Cartão
contratacoes	valor	≥ 0
pagamento	status	pendente, pago, falhou, cancelado, estornado
pagamento	valor	≥ 0
conversa	status	aberta, fechada, arquivada
mensagem	remetente_tipo	cliente, prestador
mensagem	status	enviada, recebida, lida
avaliacao	nota	Entre 1 e 5 (BETWEEN 1 AND 5)
servicos	preco	≥ 0

Tabela 14 — Restrições de domínio (CHECK constraints)

4.2.2 Integridade Referencial

Todas as relações entre entidades são implementadas com FOREIGN KEY explícitas. A política de cascadeamento adotada é ON DELETE CASCADE para as

entidades dependentes (cliente, prestador, contratacoes, pagamento, conversa, mensagem, avaliacao, prestador_especialidade), garantindo que a exclusão de um registro pai remova automaticamente todos os registros filhos associados. A exceção é a relação entre servicos e categoria, que utiliza ON DELETE RESTRICT, impedindo a exclusão de uma categoria enquanto houver serviços a ela vinculados.

4.2.3 Unicidade

Os campos email e cpf da entidade usuario possuem restrição UNIQUE, garantindo que não existam dois usuários com o mesmo endereço de e-mail ou CPF. Da mesma forma, o campo contratacao_id nas entidades conversa e avaliacao é declarado como UNIQUE, assegurando que cada contratação possua no máximo uma conversa e uma avaliação associadas.

4.3 Funções e Triggers

O banco de dados implementa funções e triggers que automatizam a atualização de campos derivados e a execução de operações recorrentes, reduzindo a dependência da camada de aplicação para manter a consistência do estado dos dados.

4.4.1 fn_atualizar_updated_at

Função genérica acionada pelos trigger de UPDATE nas tabelas usuario, contratacoes, pagamento e conversa. Atualiza o campo updated_at com o instante corrente (CURRENT_TIMESTAMP) antes de cada modificação de registro.

```
CREATE FUNCTION fn_atualizar_updated_at()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = CURRENT_TIMESTAMP;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Código 1 — Função fn_atualizar_updated_at

4.4.2 fn_atualizar_media_prestador

Acionada após INSERT, UPDATE ou DELETE na tabela avaliação. Recalcula a média aritmética das notas do prestador afetado e atualiza o campo media_avaliacao na tabela prestador. O uso de COALESCE garante que um prestador sem avaliações tenha sua média definida como 0.

```
CREATE FUNCTION fn_atualizar_media_prestador()  
RETURNS TRIGGER AS $$  
BEGIN  
    UPDATE prestador  
    SET media_avaliacao = (  
        SELECT COALESCE(ROUND(AVG(nota)::NUMERIC, 2), 0)  
        FROM avaliacao  
        WHERE prestador_id = OLD.prestador_id  
    )  
    WHERE OLD.prestador_id = OLD.prestador_id;  
END;  
$$ LANGUAGE plpgsql;
```



```

        WHERE      prestador_id      =      COALESCE (NEW.prestador_id,
OLD.prestador_id)
    )
    WHERE user_id = COALESCE (NEW.prestador_id, OLD.prestador_id);
    RETURN COALESCE (NEW, OLD);
END;
$$ LANGUAGE plpgsql;

```

Código 2 — Função fn_atualizar_media_prestador

4.4.3 fn_atualizar_total_clientes_prestador

Acionada após INSERT, UPDATE ou DELETE na tabela contratacoes. Recalcula o total de clientes distintos que possuem contratacoes com status 'Concluída' para o prestador afetado, atualizando o campo total_clientes na tabela prestador.

```

CREATE FUNCTION fn_atualizar_total_clientes_prestador()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE prestador
    SET total_clientes = (
        SELECT COUNT(DISTINCT cliente_id)
        FROM contratacoes
        WHERE      prestador_id      =      COALESCE (NEW.prestador_id,
OLD.prestador_id)
        AND status = 'Concluída'
    )
    WHERE user_id = COALESCE (NEW.prestador_id, OLD.prestador_id);
    RETURN COALESCE (NEW, OLD);
END;
$$ LANGUAGE plpgsql;

```

Código 3 — Função fn_atualizar_total_clientes_prestador

4.4.4 finalizar_contratacao

Procedimento armazenado que encapsula a lógica de finalização de uma contratação: atualiza o status para 'Concluída', registra o instante de conclusão e atualiza o campo `updated_at`. Deve ser invocado pela camada de aplicação ao confirmar a conclusão de um serviço.

```
CREATE FUNCTION finalizar_contratacao(p_contratacao_id INT)
RETURNS VOID AS $$
BEGIN
    UPDATE contratacoes
    SET status = 'Concluída',
        conclusao = CURRENT_TIMESTAMP,
        updated_at = CURRENT_TIMESTAMP
    WHERE contratacao_id = p_contratacao_id;
END;
$$ LANGUAGE plpgsql;
```

Código 4 — Função `finalizar_contratacao`

4.5 Views

A view `vw_resumo_contratacoes` agrega em uma única consulta as informações mais relevantes de uma contratação, incluindo o nome do cliente, o nome do prestador, o serviço contratado e a categoria. Seu objetivo é simplificar as consultas recorrentes da camada de aplicação, evitando a repetição de JOINS complexos.

```
CREATE VIEW vw_resumo_contratacoes AS
SELECT
    c.contratacao_id,      c.titulo,      c.status,      c.valor,
    c.forma_pagamento,
    uc.nome AS cliente,
    up.nome AS prestador,
    s.nome_servico,
    cat.nome_categoria AS categoria
FROM contratacoes c
JOIN usuario uc ON uc.user_id = c.cliente_id
JOIN usuario up ON up.user_id = c.prestador_id
JOIN servicos s ON s.servico_id = c.servico_id
```

```
JOIN categoria cat ON cat.categoria_id = s.categoria_id;
```

Código 5 — View vw_resumo_contratacoes

4.6 Índices

Foram criados índices adicionais sobre as colunas mais frequentemente utilizadas em cláusulas WHERE e JOIN, com o objetivo de otimizar o desempenho das consultas de maior volume. A Tabela 15 lista os índices implementados.

Índice	Tabela	Coluna	Justificativa
idx_usuario_tipo	usuario	tipo_usuario	Filtro por perfil de usuário
idx_servico_categoria	servicos	categoria_id	Busca por categoria
idx_servico_prestador	servicos	prestador_id	Serviços de um prestador
idx_contratacao_cliente	contratacoes	cliente_id	Histórico do cliente
idx_contratacao_prestador	contratacoes	prestador_id	Histórico do prestador
idx_contratacao_servico	contratacoes	servico_id	Contratações por serviço
idx_pagamento_contratacao	pagamento	contratacao_id	Pagamentos de uma contratação

Tabela 15 — Índices do banco de dados

5 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: mar. 2026.

NESTJS. NestJS — A progressive Node.js framework. Disponível em: <https://nestjs.com>. Acesso em: mar. 2026.

STRIPE, INC. Stripe API Reference. Disponível em: <https://stripe.com/docs/api>. Acesso em: mar. 2026.

AMAZON WEB SERVICES. Amazon RDS for PostgreSQL. Disponível em: <https://aws.amazon.com/rds/postgresql/>. Acesso em: mar. 2026.